

Домашнее задание 1: подготовка стенда на Windows

Выполняйте эту инструкцию **после** занятия «Семинар 1. Настройка S3 и колоночные форматы: подготовка источника данных», как этап 0 домашней работы. На семинаре команды выполняет преподаватель; студентам заранее устанавливать или запускать стенд не нужно.

Здесь вы установите зависимости и проверите доступность среды. После этого переходите к [воспроизведению учебного примера в ДЗ](#), затем — к своему открытому источнику.

Рекомендуемый вариант

WSL 2 — среда Linux внутри Windows; в инструкции используем дистрибутив Ubuntu.

Docker-контейнер — изолированный процесс с зависимостями, образ — шаблон его запуска, Compose — средство запуска связанных сервисов по файлу конфигурации. Volume — постоянное хранилище контейнера.

Используйте Docker Desktop с backend WSL 2 и запускайте команды внутри Ubuntu/WSL. Это даёт одинаковые Linux-команды на macOS и Windows и избегает медленного bind mount из C:\....

Compose поднимает MinIO, Spark 3.5.9, PostgreSQL 16 и Trino 483. Spark использует библиотеку Iceberg 1.11.0; отдельного сервиса Iceberg нет. Java/Spark в Windows устанавливать не нужно. Python 3 нужен внутри WSL для генерации CSV.

Стенд учебный и локальный. Не открывайте его порты во внешнюю сеть.

Требования

По документации Docker Desktop на момент актуализации:

- поддерживаемая текущим Docker Desktop версия Windows 11 64-bit; совместимость Windows 10 отдельно сверяйте с актуальными требованиями Docker и поддержкой Microsoft;
- WSL 2.1.5 или новее; лучше обновить до последней версии;
- аппаратная виртуализация включена в BIOS/UEFI;
- минимум 8 ГБ RAM, комфортно 12–16 ГБ;
- 15 ГБ свободного места;
- интернет для первого build.

Если ноутбук не позволяет выделить указанные ресурсы, обсудите с преподавателем доступную учебную машину или иной способ выполнения. Работа остаётся индивидуальной.

Актуальные требования всегда сверяйте с [Docker Desktop for Windows](#): поддерживаемые сборки Windows меняются вместе с циклом поддержки Microsoft.

1. Установите или обновите WSL

PowerShell от имени администратора:

```
wsl --install -d Ubuntu
wsl --update
wsl --version
```

После первой команды может потребоваться перезагрузка. Откройте Ubuntu и создайте Linux-пользователя.

Если `wsl --install` недоступна, используйте официальную инструкцию Microsoft для вашей версии Windows, а не случайный сторонний скрипт.

2. Установите Docker Desktop

Скачайте [Docker Desktop for Windows](#).

В Docker Desktop:

1. Settings → General → Use WSL 2 based engine;
2. Settings → Resources → WSL Integration → включить Ubuntu;
3. дождаться статуса Running.

Проверка в Ubuntu/WSL:

```
docker version
docker compose version
docker run --rm hello-world
```

Не устанавливайте второй Docker Engine внутри Ubuntu поверх интеграции Docker Desktop: параллельные daemon/context часто становятся источником путаницы.

3. Выделите ресурсы

Для стенда желательно 4 CPU и 6–8 ГБ памяти. В современном WSL память выделяется динамически. Если раньше создавали `%UserProfile%\wslconfig` с жёстким малым лимитом, проверьте его.

После изменения настроек WSL:

```
wsl --shutdown
```

Затем снова запустите Docker Desktop и Ubuntu.

4. Установите Python 3 в Ubuntu

```
sudo apt update
sudo apt install -y python3
python3 --version
```

5. Храните проект в Linux-файловой системе

Рекомендуемый путь:

```
mkdir -p ~/projects
cd ~/projects
```

Скачайте `lecture_01_student.zip` по ссылке преподавателя. Распакуйте и перенесите папку `lecture_01_student` в `projects` внутри Ubuntu/WSL. Например, откройте `\\wsl.localhost\Ubuntu\home\<user>\projects` в Проводнике Windows и скопируйте распакованную папку. Имя дистрибутива проверьте командой `wsl -l -v`. Репозиторий преподавателя не нужен. Работайте по пути вида:

```
/home/<user>/projects/lecture_01_student
```

Не запускайте data-heavy часть из `/mnt/c/...`, если наблюдаете медленное чтение: bind mounts между Windows и Linux обычно медленнее, чем файлы внутри WSL.

6. Проверьте и соберите стенд

В Ubuntu:

```
cd ~/projects/lecture_01_student/infra
docker compose config --quiet
docker compose build
```

Замените путь на путь к своей копии. Все дальнейшие команды `docker compose` и команды учебного примера выполняйте из `infra/` в Ubuntu/WSL. Первый build скачивает образы и Maven-зависимости; не переходите к следующему шагу, пока он не завершится успешно.

7. Запустите сервисы

```
docker compose up -d
docker compose ps --all
```

Дождитесь готовности PostgreSQL и MinIO. Сервис `minio-init` создаёт бакеты и завершается: статус `Exited (0)` для него нормален. В студенческом комплекте имя проекта — `hse-lakehouse-student`; имена контейнеров Compose создаёт автоматически. Остальные четыре сервиса должны оставаться запущенными; `Exited` с ненулевым кодом или постоянный `Restarting` требуют диагностики ниже.

Минимальные проверки среды:

```
docker compose exec spark python3 -c "import pyspark; print(pyspark.__version__)"
docker compose exec trino trino --execute "SHOW CATALOGS"
```

Версия PySpark должна совпасть с версией в `infra/spark/Dockerfile` (сейчас 3.5.9); в Trino должны быть видны `lakehouse` и `memory`. Если Trino ещё запускается, проверьте его логи и повторите запрос после сообщения о готовности.

Открывайте из браузера Windows:

Сервис	Адрес	Доступ
MinIO Console	http://localhost:9001	admin / hse2026minio
MinIO S3 API	http://localhost:9000	для скриптов
Trino UI	http://localhost:8088	без пароля, только локально

Откройте MinIO Console и убедитесь, что есть бакеты `raw` и `datalake`. На этом проверка установки закончена: данные и таблицы ещё не создавались. Перейдите к [этапу 0 ДЗ](#) и выполните последовательность учебного примера один раз. Она включает генерацию небольшого CSV на 200 000 строк; отдельный большой синтетический набор не нужен.

SQL-клиент для занятия и домашней работы

На семинаре преподаватель работает с Trino через DataGrip. Подключение описано в [отдельной инструкции](#): `localhost:8088`, пользователь `teacher`, пароль пустой, каталог `lakehouse`, схема `dwh`. Встроенный сайт Trino показывает выполнение запросов; DataGrip предоставляет SQL-редактор и табличный результат. В DataGrip можно просматривать и выполнять сохранённый `infra/trino/scripts/seminar_demo.sql` по блокам.

Для ДЗ DataGrip необязателен: тот же SQL выполняется через CLI. Выбор клиента не влияет на 10-балльную оценку. Устанавливать новый клиент во время семинара не требуется. После перезапуска Trino восстановите справочник из блока 5 SQL-файла.

Preview CSV, JSON и Parquet в установленной MinIO Console не поддерживается. В MinIO проверяйте ключ, размер и список объектов; CSV/JSON открывайте в редакторе, а строки Parquet/Iceberg показывайте через читающий их движок.

Автозапуск учебных контейнеров в Compose отключён (`restart: "no"`). После запуска Docker Desktop вручную выполните `docker compose up -d` из `infra/`.

DataGrip устанавливается в Windows; параметры JDBC остаются теми же. Shell-команды из инструкции выполняйте в Ubuntu/WSL, используя свой путь к `infra/`. Команда `open -a Docker` относится только к macOS: в Windows откройте Docker Desktop через меню «Пуск».

Диагностика

Проверка WSL и Docker:

```
wsl --status
wsl --version
docker context ls
```

Логи в Ubuntu:

```
docker compose ps --all
docker compose logs --tail=100 postgres
docker compose logs --tail=100 minio
docker compose logs --tail=100 minio-init
docker compose logs --tail=100 spark
docker compose logs --tail=100 trino
```

Если `docker` не найден в Ubuntu, проверьте WSL Integration в Docker Desktop и перезапустите WSL:

```
wsl --shutdown
```

Если не открывается `localhost`, сначала проверьте `docker compose ps` и занятые порты:

```
Get-NetTCPConnection -LocalPort 9000,9001,8088 -ErrorAction SilentlyContinue
```

Остановка и очистка

```
docker compose down
```

Полностью удалить учебные volumes:

```
docker compose down -v
```

`-v` безвозвратно удалит данные MinIO и PostgreSQL этого compose-проекта. Не используйте команду в другом каталоге по аналогии, не проверив имя проекта.

Источники

- [Docker Desktop for Windows — установка и требования](#)
- [Docker Desktop WSL 2 backend](#)
- [WSL 2 best practices](#)
- [Apache Spark releases](#)
- [Apache Iceberg releases](#)
- [Trino releases](#)